1 Описание механики игры

Это небольшая текстовая игра, в которой два персонажа сражаются друг с другом по очереди. У каждого персонажа есть имя, количество очков здоровья (жизней) и сила удара.

Игра проходит в виде пошагового боя. На каждом ходу один персонаж атакует другого, а затем — наоборот. Удар уменьшает здоровье противника. Игра заканчивается, когда здоровье одного из персонажей достигает нуля или становится отрицательным. Побеждает тот, кто остаётся в живых.

В этой игре:

- участвуют два типа персонажей: игрок и враг;
- каждый персонаж имеет имя, здоровье и силу удара;
- при атаке здоровье противника уменьшается;
- победитель определяется автоматически;
- весь вывод — текстовый (в консоли).

Цель проекта — реализовать эту механику, параллельно изучая базовые концепции объектно-ориентированного программирования.

---

2 Этапы разработки

Каждый этап вводит новые принципы ООП: классы, методы, наследование, инкапсуляцию и другие.

Этап 1. Базовый класс `Character`

Создаётся основной класс, представляющий любого персонажа в игре.

Поля (приватные, в стиле `camelCase`):

- `private string name` — имя персонажа, отображается при выводе и в ходе боя;
- `private int hp` — очки здоровья. Уменьшаются при получении урона. Если значение ≤ 0, персонаж считается побеждённым;
- `private int power` — сила удара. Это число, на которое будет уменьшаться здоровье противника при атаке.

Конструктор и методы:

- `Character(string name, int hp, int power)` — получает начальные характеристики персонажа и сохраняет их в соответствующих полях;
- `Attack(Character target)` — получает объект другого персонажа и уменьшает его здоровье. Также выводит в консоль текст атаки;
- `GetStatus()` — показывает имя персонажа и его текущее здоровье. Используется для отображения состояния перед или после атаки.

Этап 2. Статические члены класса

Теперь класс отслеживает общее количество созданных персонажей. Также добавляются методы, не привязанные к конкретному объекту.

Дополнительное поле:

- `private static int count` — статическое поле (общее для всех объектов). В конструкторе `Character` его значение увеличивается на единицу при создании каждого нового персонажа.

Статические члены:

- `HowMany()` — статический метод, возвращает количество созданных персонажей. Вызывается через класс: `Character.HowMany();`
- `IsValidStats(int hp, int power)` — статический метод, который проверяет корректность переданных характеристик, например что здоровье и сила удара положительны. Такой метод не относится к конкретному персонажу и не использует `this`.

Этап 3. Наследование: `Player` и `Enemy`

Добавляются подклассы, унаследованные от `Character`. У игрока и врага могут быть разные стили атаки (например, разный текст), но логика остаётся общей.

Новые классы:

- `Player : Character` — подкласс для главного героя;
- `Enemy : Character` — подкласс для противника.

Особенности:

- классы наследуют публичные и защищённые члены `Character`; приватные поля базового класса напрямую из наследников недоступны;
- общие данные и базовая логика по-прежнему остаются в `Character`, чтобы не копировать их в `Player` и `Enemy`;
- конструктор базового класса вызывается в объявлении конструктора наследника через `: base(...)`.

Этап 4. Инкапсуляция (скрытие данных)

Чтобы нельзя было напрямую изменять здоровье персонажа извне, его изменение ограничивается. Для чтения значения используется свойство, а изменение выполняется контролируемо внутри класса.

Изменения:

- приватные поля `name`, `hp` и `power` остаются скрытыми от внешнего кода;
- для чтения значений добавляются публичные свойства, например `public string Name => name;` и `public int Hp => hp;`;
- `Attack(Character target)` больше не изменяет `target.hp` напрямую, а вызывает публичный метод самого объекта-цели: `target.TakeDamage(power)`.

Новые методы:

- `TakeDamage(int damage)` — контролируемо уменьшает здоровье персонажа и не позволяет внешнему коду напрямую присваивать `hp`;

- `IsAlive()` — экземплярный метод, возвращающий `true`, если здоровье больше нуля.

Уже существующий `GetStatus()` теперь получает данные через свойства `Name` и `Hp`.

Этап 5. Полиморфизм (один интерфейс — разные классы)

Теперь можно обращаться к разным объектам одинаково, не заботясь о том, какого они класса. Это удобно при выполнении атаки в цикле.

Изменения:

- метод `Attack(Character target)` в `Character` объявляется с `virtual`;
- `Player` и `Enemy` переопределяют его с помощью `override`, сохраняя общий интерфейс, но меняя конкретное поведение или текст атаки;
- все персонажи имеют общий набор методов `Attack()`, `GetStatus()` и `IsAlive()`;
- объекты `Player` и `Enemy` можно хранить в одной коллекции `List<Character>` и вызывать `Attack()` через ссылку типа `Character`; будет выполнена реализация реального типа объекта.

Пример поведения:

- игрок и враг по очереди атакуют друг друга;
- после каждой атаки вызывается `GetStatus()` для отображения текущего состояния;
- после каждой атаки проверяется `IsAlive()`.

Этап 6. Переопределение стандартных методов `object`

Добавляются стандартные методы C#, чтобы объекты персонажей вели себя более естественно при выводе и сравнении.

Изменения:

- вместо отдельного формирования строки внутри `GetStatus()` переопределяется `ToString()`. Он возвращает строку вида `"Игрок (HP: 5)"`, поэтому состояние можно выводить обычным `Console.WriteLine(character)`;
- после этого `GetStatus()` можно удалить как лишний метод: его задачу выполняет стандартный механизм `ToString()`;
- `Equals(object? obj)` позволяет задать логическое сравнение персонажей. Обычно сравниваются имена или ID персонажа;
- при переопределении `Equals()` следует также переопределить `GetHashCode()`. Если требуется именно сравнение с помощью операторов `==` и `!=`, их нужно перегрузить отдельно.

Этап 7. Абстрактный базовый класс (необязательно)

Если хочется повысить строгость, класс `Character` можно сделать абстрактным, то есть запретить его непосредственное использование.

Для этого:

- класс объявляется как `abstract class Character`;
- существующая реализация `Attack(Character target)` в `Character` убирается, а сам метод вместо `virtual` объявляется как `public abstract void Attack(Character target);`;
- реализации `Attack()` в `Player` и `Enemy` остаются и по-прежнему помечаются `override`.

Результат:

- нельзя будет создать объект `Character` напрямую с помощью `new Character(...)`;
- все неабстрактные наследники обязаны будут реализовать метод `Attack()`.